

Bug report — @bug tagged cases

Printable copy: [bug-report.pdf](#) .

Product: Rethink Clinical

Environment: <https://clinical.dev2.rethinkbhtech.com>

API: <https://dev2.internal.rethinkbhtech.com/mobile-gateway-api>

Reported by: AIDLC E2E suite (Playwright + Cucumber)

Date: 2026-08-27

Suite command: `npx cucumber-js -p bugs`

These reports cover only scenarios tagged `@bug` . Each scenario asserts **correct** product behaviour. They are excluded from `npm test` so the default suite stays green. They are expected to **pass** once the defect is fixed; then the `@bug` tag should be removed.

ID	Title	Severity	Area	Automated case	Default run
DEF-1	Chart grouping select does not regroup the chart	Medium	Analyze Data	AZ-7	Fail (held out)
DEF-2	behaviorplans returns 500 for every client	High	Behavior Support / API	BS-2	Fail (held out)
DEF-4	automastery-evaluations 500 under concurrent load	Medium	Analyze Data / API	AZ-6b	Intermittent (held out)
DEF-5	Server error shown as “no behavior plans yet”	High	Behavior Support UI	BS-3	Fail (held out)

Related (not `@bug` tagged): **DEF-3** — `staff-role` returns credential fields. Mitigated in the harness only (`support/scrub.ts`). See `docs/defects.md` .

DEF-1 — Chart grouping does not regroup the chart

Field	Detail
Bug ID	DEF-1
Title	Analyze Data grouping select updates its label but the chart stays grouped by Domain
Severity	Medium
Priority	P2 (control looks functional; data is silently wrong)
Module	Analyze Data — Mastered Targets report
Build / env	clinical.dev2, 2026-08-27
Reproducibility	Always (verified in a real browser and in the Playwright harness)
Test case	AZ-7 — <i>Changing the grouping regroups the chart</i>
Feature	<code>features/analyze-data/analyze-data.feature</code>

Tags

@ui @api @analyze-data @bug

Description

On `/clients/{id}/analyze-data`, the grouping control (`data-testid="mastered-grouping-select"`) offers Domain, Category, and Area. Choosing **Category** (or Area) changes the select's displayed label. The Highcharts skill-area chart does not regroup: the x-axis categories remain the program **domains**, and Highcharts' accessible description still reads *"The chart has 1 X axis displaying Domain"*.

Preconditions

1. Signed-in clinician (dev login is sufficient).
2. A client with programs that have more than one distinct `category` (the resolved test client does).

Steps to reproduce

1. Open a client's **Analyze Data** page.
2. Confirm the chart x-axis lists domains (for example Adaptive Living, Safety, junk domains such as E2EDomain if present).
3. Open **Grouping** and select **Category**.
4. Wait several seconds for any redraw.
5. Read the x-axis labels and the chart's "X axis displaying ..." text.

Expected result

- The grouping select shows **Category**.
- The chart x-axis categories are the client's distinct program **categories** (covering those returned by `GET /clinical/v1/clients/{id}/programs`).
- Those categories differ from the previous Domain grouping.
- The chart description refers to Category, not Domain.

Actual result

- The select shows **Category**.
- The x-axis is unchanged (same domain labels as before).
- Description still says **Domain**.

Evidence

- Manual inspection on client `892745` .
- Harness probe: before/after Category, identical 13 domain labels; a11y text still *"X axis displaying Domain"*.

Automated assertion (should pass when fixed)

1. Open Analyze Data for the resolved client.
2. Group the chart by `"Category"` .
3. Chart categories cover the client's distinct program categories.
4. Chart categories differ from the snapshot taken under Domain grouping.

Impact

Clinicians can believe they are viewing mastery by category or area while still seeing domain grouping. Decisions based on the chart can be wrong without any error message.

Suggested direction

Wire `mastered-grouping-select` to the same data transform that feeds Highcharts (or rebuild the series when the select changes). Do not update the label unless the chart data has regrouped.

Workaround in the suite

AZ-7b still asserts that Domain, Category, and Area are **offered**, so the control is not untested.

DEF-2 — `behaviorplans` returns HTTP 500 for every client

Field	Detail
Bug ID	DEF-2
Title	GET .../observations/v1/client/{id}/behaviorplans returns 500 for all clients
Severity	High
Priority	P1
Module	Behavior Support API (observations)
Build / env	clinical.dev2, 2026-08-27
Reproducibility	Always (all 24 clients in the crawl / live list)
Test case	BS-2 — <i>Behavior plans returns the client's plans</i>
Feature	features/behavior-support/behavior-support.feature
Tags	@api @behavior-support @bug

Description

The Behavior Support page depends on:

```
GET {API_BASE}/observations/v1/client/{clientId}/behaviorplans
```

This call returns **HTTP 500** for every client checked. The UI retries once, then shows an unavailable notice. Plan counts render as zero.

Preconditions

Authenticated request with a valid bearer token (same as other clinical APIs).

Steps to reproduce

- 1. Authenticate (harvested session or equivalent).
- 2. GET `/observations/v1/client/{anyValidClientId}/behaviorplans` .
- 3. Repeat for additional client ids if needed.

Expected result

HTTP **200** and a body describing the client’s behavior plans (empty list is acceptable if the client truly has none).

Actual result

HTTP **500** for every client id tried.

Evidence

- Crawl and live calls on 2026-08-27: status 500 for all 24 clients.
- UI network: two 500s per page load (initial + retry) on client 892745 .

Automated assertion (should pass when fixed)

1. Request the resolved client’s behavior plans.
2. Response status is **200**.

Impact

Behavior Support cannot show real plans. Combined with DEF-5, the UI also **misrepresents** the failure as an empty caseload.

Suggested direction

Fix the observations service / gateway so the endpoint returns a normal envelope. Do not treat 500 as the contract in tests.

Workaround in the suite

BS-1 only asserts that the rail and novel-behaviors **chrome** render (true whether the API is up or down).

DEF-4 — automastery-evaluations returns 500 under concurrent load

Field	Detail
Bug ID	DEF-4
Title	Analyze Data fan-out causes intermittent 500 (sometimes a raw .NET exception body)
Severity	Medium
Priority	P2 (intermittent; report can under-count pending mastery)
Module	Analyze Data load / automastery-evaluations
Build / env	clinical.dev2, 2026-08-27
Reproducibility	Intermittent under page-load concurrency; not reproduced with sequential calls
Test case	AZ-6b — <i>Every request the report makes succeeds</i>
Feature	features/analyze-data/analyze-data.feature
Tags	@network @analyze-data @bug

Description

Opening Analyze Data issues about **two HTTP calls per program** (targets and automastery-evaluations? status=flagged). On a client with ~20 programs that is ~40 concurrent XHRs.

Under that load, the backend **sometimes** returns 500. At least one captured body was a .NET exception string (System.Inv...) rather than JSON.

Sequential probing (60 requests, three passes over 20 programs) all returned **200**. The failure correlates with **concurrency**, not a specific program id.

Preconditions

- 1. Signed-in clinician.
- 2. Client with many programs (e.g. the shared resolved client).

Steps to reproduce

- 1. Open `/clients/{id}/analyze-data` and record network (DevTools or Playwright).
- 2. Repeat page loads if the first load is clean.
- 3. Look for `.../programs/{id}/automastery-evaluations?status=flagged` with status 500.

Expected result

Every per-program `targets` and `automastery-evaluations` request returns status **< 400**. JSON bodies parse.

Actual result

Occasional **500**. Example from a full suite run:

`GET .../programs/256/automastery-evaluations?status=flagged` → 500

Evidence

- Suite failure during Unit 4: 500 on program 256 under page fan-out.
- Sequential 60-call check: all 200.
- JSON parse error `Unexpected token 'S'` when the body was `System.Inv...`

Automated assertion (should pass when fixed)

- 1. Record traffic while opening Analyze Data.
- 2. None of the report’s per-program requests failed (status $\geq$ 400).

Note: Because the bug is intermittent, AZ-6b may **pass** on some runs. Treat it as a monitor, not a one-shot gate.

Impact

Pending mastery review can miss flagged items. The page may look complete while some evaluations never loaded.

Suggested direction

- 1. Make the evaluations endpoint safe under concurrent identical-scope traffic (or serialize/queue).
- 2. Return JSON error envelopes, not raw exception text.
- 3. Consider aggregating targets/evaluations server-side so the report is not an N+1 fan-out (~40 round trips).

Workaround in the suite

AZ-6 still asserts that the **requests are made**, without requiring every status to be 200.

DEF-5 — Server error presented as an empty plan list

Field	Detail
Bug ID	DEF-5

Title	Behavior Support shows “no behavior plans yet” and “data is unavailable” together
Severity	High (trust / clinical safety of the message)
Priority	P1
Module	Behavior Support UI
Build / env	clinical.dev2, 2026-08-27
Reproducibility	Always while DEF-2 holds, after the retry completes
Test case	BS-3 — <i>The page does not claim there are no plans while data is unavailable</i>
Feature	features/behavior-support/behavior-support.feature
Tags	@ui @behavior-support @bug
Depends on	DEF-2 (500) triggers the unavailable path

Description

When `behaviorplans` fails, the page eventually shows **both**:

- Plan rail: `Current (0) · Inactive (0) · This client has no behavior plans yet.`
- Banner: **Behavior support data is unavailable right now. Try again shortly.**

Zero counts and the empty-state sentence are derived from a **failed** request, not from a successful empty list. A clinician scanning the rail can conclude the client has no plans.

The unavailable banner appears only after the **second** (retry) 500. Checking too early sees only the empty state.

Preconditions

DEF-2 still failing. Authenticated user. Any client (all currently 500).

Steps to reproduce

1. Open `/clients/{id}/behavior-support`.
2. Wait until network is quiet (two `behaviorplans` 500s, ~1–2 seconds after the first).
3. Read the plan rail and the page footer/banner.

Expected result

Only one of these can be true:

- Data loaded successfully and the client has no plans → empty state, **no** unavailable banner.
- Data failed to load → unavailable banner, **no** confident “no behavior plans yet” / zero counts presented as fact.

Actual result

Both empty-state copy **and** the unavailable notice are visible (`plan-rail-empty` and `behavior-support-unavailable`).

Evidence

Live DOM on client 892745 : both nodes `display:block` , `visibility:visible` , non-zero size. Full page text includes both sentences.

Automated assertion (should pass when either DEF-2 is fixed *or* the UI is corrected)

1. Open Behavior Support and wait until `behaviorplans` traffic stops.
2. Empty plan message and unavailable notice are **not both** shown.

This invariant stays valid after a backend fix: a true empty caseload should not show “unavailable”.

Impact

High for trust. Staff may skip writing or reviewing plans because the rail says there are none.

Suggested direction

If the request fails (after retry): show only the unavailable/error state; do not bind `0` / “no plans yet” from a failed payload. If the request succeeds with zero plans: show empty state only.

How to re-run these reports

```
npx cucumber-js -p bugs
```

Expected today: **four scenarios, three or four failures** (AZ-6b may pass on a lucky load). After a fix, the matching scenario should go green; remove `@bug` and fold it into `npm test` .

Full write-up of anomalies that are **not** `@bug` cases: `docs/defects.md` .